KVShuttle: A Comprehensive Benchmark and Decision Framework for KV Cache Transfer Compression in Disaggregated LLM Serving

Awneesh Tiwari *awneeshtiwari@gmail.com*
Independent Researcher

Abstract

Disaggregated prefill/decode serving separates LLM inference across GPUs, creating a KV cache transfer bottleneck between them. While numerous KV cache compression methods exist, they were designed for memory reduction—not transfer optimization—and have never been systematically compared under transfer-realistic conditions. I present **KVShuttle**, the first comprehensive benchmark evaluating 14 compression strategies across 6 models (3B–8B parameters), measuring the full compress–transfer–decompress pipeline with GPU-calibrated timing on Tesla T4 and A100 GPUs. From over 12,000 experimental data points spanning quantization, structured coding, low-rank projection, token pruning, lossless, and hybrid methods, I find that: (1) the optimal strategy depends strongly on network bandwidth and model architecture—no single method dominates across all regimes; (2) without GPU-accelerated kernels, compression only helps below 1 Gbps; with GPU pipelining, the break-even extends to 50–100 Gbps, covering most datacenter Ethernet networks; (3) cosine similarity is a poor predictor of generation quality ($\rho = 0.86$ with token agreement, $\rho = -0.69$ with perplexity), with INT4 achieving 0.99 cosine yet producing near-random output on some models; (4) model sensitivity varies dramatically, with Qwen models suffering catastrophic failure under INT4 (<9% token agreement) while Llama/Mistral maintain >95%; (5) an oracle compressor selector achieves up to 2.1× speedup over any fixed strategy by adapting to per-request conditions. Based on these findings, I provide a practitioner decision framework, automated compressor routing, and release the complete benchmark toolkit at `https://github.com/awneesht/KVShuttle`.

1 Introduction

Disaggregated prefill/decode serving (Zhong et al., 2024; Patel et al., 2024) has become the standard architecture for production LLM inference, separating compute-intensive prefill from memory-bound decode onto specialized GPU pools. This separation introduces a new bottleneck: the KV cache computed during prefill must be transferred to the decode GPU before generation can begin. For a Llama-3.1-8B model with a 2048-token prompt, the KV cache is 128 MB in FP16. At 10 Gbps Ethernet, this transfer takes over 100 ms—comparable to the prefill computation itself.

As prompt lengths grow with long-context applications and retrieval-augmented generation (RAG), KV cache transfer increasingly dominates end-to-end latency. Numerous KV cache compression methods have been proposed (Liu et al., 2024c;a; Hooper et al., 2024), but they were designed to reduce *memory footprint* during inference, not to optimize *network transfer*. A method that halves KV cache size is only beneficial for transfer if its compression and decompression overhead is less than the transfer time saved—a condition that depends on network bandwidth, GPU compute speed, and whether compression can be pipelined with transfer.

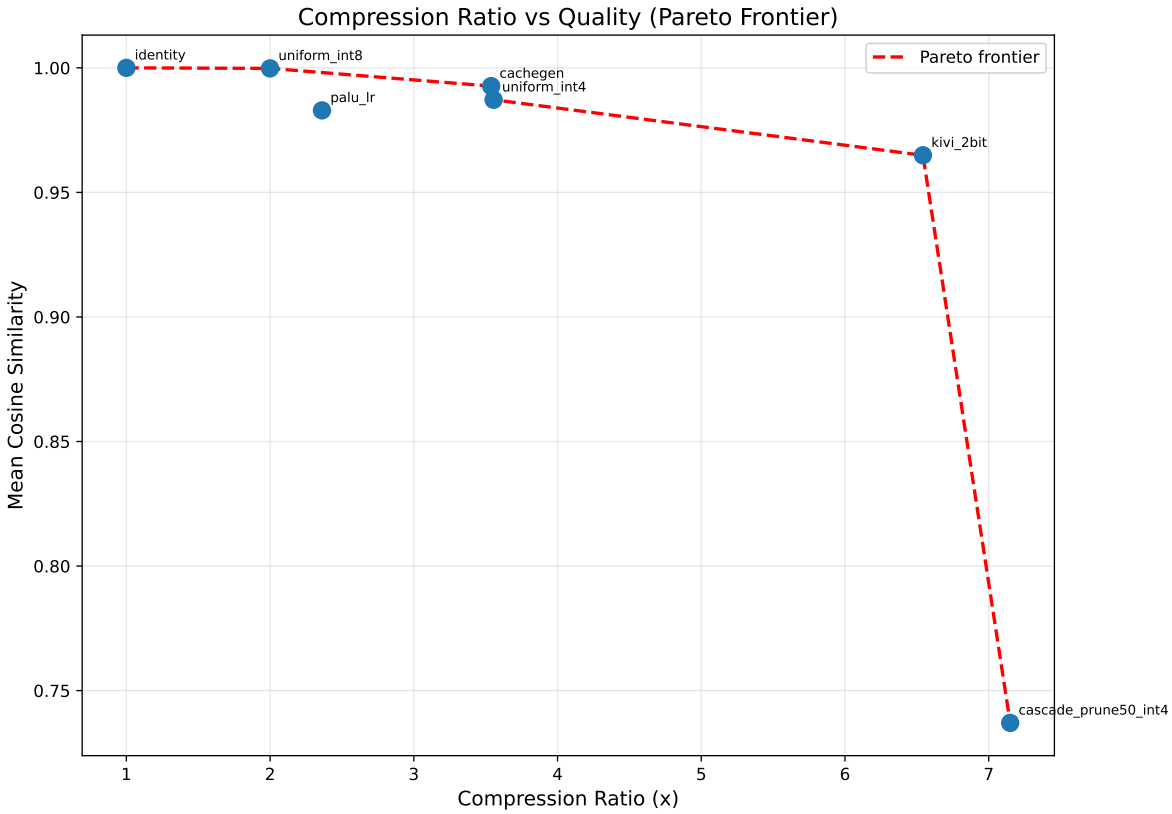


Figure 1: Pareto frontier of compression ratio vs. key cosine similarity across all 14 strategies. Methods above the Pareto front offer strictly dominated trade-offs. INT8 provides the best quality-per-compression, while cascade and KIVI push the compression frontier at quality cost.

No prior work systematically evaluates compression methods through this transfer lens. Existing benchmarks (Yuan et al., 2024; Jegou et al., 2025) measure quality and memory but not transfer cost; HACK (Yao et al., 2025) accounts for compress/decompress overhead but compares only three quantization methods. I address this gap with KVShuttle, contributing:

1. **A unified benchmark** of 14 compression strategies spanning six families—quantization, structured coding, low-rank projection, token pruning, lossless, and hybrid methods—evaluated across 6 models (3B–8B parameters) with over 12,000 experimental data points (Section 3).

2. **Transfer pipeline analysis** with an analytical model validated against real TCP measurements, showing that pipelining compress/transfer/decompress reduces latency by 36% on average (Section 4).

3. **GPU-calibrated timing** with zero-copy CUDA Event measurement on Tesla T4 and A100 GPUs, demonstrating 34–355× speedup over CPU baselines and fundamentally shifting the break-even bandwidth (Section 6).

4. **End-to-end generation quality metrics** revealing that cosine similarity ($\rho = 0.86$ Spearman with token agreement, $p < 10^{-3}$) is an inadequate quality proxy, with Wilcoxon signed-rank tests confirming statistically significant ($p < 10^{-9}$) quality differences between seemingly similar compressors (Section 5).

5. **Automated compressor selection** via oracle and learned routers that adapt to bandwidth, model architecture, and quality requirements, achieving up to 2.1× speedup over fixed strategies (Section 7).

Table 1: KV cache size (MB, FP16) by model architecture and sequence length. Models with fewer GQA heads (Qwen: 2–4) have smaller caches but, as we show, are more sensitive to compression errors.

Model	L	H_{kv}	d	**256**	**512**	**1024**	**2048**
Qwen2.5-3B	36	2	128	4.5	9.0	18.0	36.0
Llama-3.2-3B	28	8	128	14.0	28.0	56.0	112.0
Phi-3.5-mini	32	32	96	48.0	96.0	192.0	384.0
Qwen2.5-7B	28	4	128	7.0	14.0	28.0	56.0
Llama-3.1-8B	32	8	128	16.0	32.0	64.0	128.0
Mistral-7B	32	8	128	16.0	32.0	64.0	128.0

6. **A practitioner decision framework** with deployment recipes, break-even analysis, and a selection flowchart (Section 8).

2 Background and Motivation

2.1 Disaggregated LLM Serving

In disaggregated serving (Zhong et al., 2024; Patel et al., 2024), prefill and decode phases run on separate GPU pools connected via a datacenter network. The prefill GPU computes the KV cache $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{L \times H \times S \times d}$ for a prompt of S tokens, where L is the number of layers, H the number of KV heads, and d the head dimension. This cache must then be serialized, transferred, and deserialized on the decode GPU before autoregressive generation begins.

Formal problem. Given a KV cache $\mathcal{C} = (\mathbf{K}, \mathbf{V})$ of size $|\mathcal{C}| = 2 \cdot L \cdot H \cdot S \cdot d \cdot b$ bytes (where b is the bytes per element, typically 2 for FP16), network bandwidth B (bytes/sec), and a compressor f with compress time $t_c(f)$, compressed size $|f(\mathcal{C})|$, and decompress time $t_d(f)$, the transfer latency is:

$$T(f, B) = t_c(f) + \frac{|f(\mathcal{C})|}{B} + t_d(f) \tag{1}$$

Compression is beneficial when $T(f, B) < T(\text{identity}, B) = |\mathcal{C}|/B$, which requires:

$$t_c(f) + t_d(f) < \frac{|\mathcal{C}| - |f(\mathcal{C})|}{B} = \frac{(1 - 1/r) \cdot |\mathcal{C}|}{B} \tag{2}$$

where $r = |\mathcal{C}|/|f(\mathcal{C})|$ is the compression ratio. This defines a *break-even bandwidth*: compression helps only when the network is slow enough that the transfer savings exceed the computational overhead.

2.2 KV Cache Sizes Across Models

Table 1 shows the dramatic variation in KV cache sizes across the 6 models in our benchmark. Cache size depends on three architectural choices: the number of layers L, the number of key-value heads H_{kv} (which differs from the attention heads in grouped-query attention, GQA), and the head dimension d. Phi-3.5-mini, despite being only 3.8B parameters, has the largest KV cache (384 MB at 2048 tokens) because it uses full multi-head attention ($H_{kv} = 32$). Qwen2.5-3B has the smallest cache (36 MB) due to aggressive GQA ($H_{kv} = 2$), but this also concentrates information in fewer heads, making each head more critical and compression more risky.

2.3 The Compression Landscape

Existing KV cache compression methods span several families:

- **Quantization:** Reduces bit-width from FP16 to INT8 (Dettmers et al., 2022), INT4, FP8, or even 2-bit (Liu et al., 2024c; Hooper et al., 2024). Per-tensor, per-channel, or per-token scaling strategies trade off granularity for metadata overhead.

Table 2: All 14 compression strategies in KVShuttle. Ratio and cosine similarity are from the compression sweep on Llama-3.1-8B (400 results per compressor across 8 bandwidth levels). Token agreement (TA) and perplexity delta (Δppl) are means over 5 models and 50 WikiText prompts each (250 total evaluations), with 95% bootstrap CIs. — indicates the compressor was not evaluated for generation quality.

Strategy	**Family**	**Ratio**	**Key** cos	**Val** cos	**TA**	**Δppl**
identity	baseline	1.0×	1.0000	1.0000	1.000	0.00
lossless_zstd	lossless	1.1×	1.0000	1.0000	—	—
lossless_lz4	lossless	1.0×	1.0000	1.0000	—	—
uniform_int8	quantization	2.0×	0.9998	0.9998	0.972	0.03
fp8_e4m3	quantization	2.0×	0.9926	0.9987	—	—
uniform_int4	quantization	3.6×	0.9874	0.9935	0.593	13,294
kivi_2bit	quantization	6.7×	0.9587	0.8692	0.737	7.50
kvquant_2bit	quantization	7.6×	0.6471	0.6554	—	—
cachegen	structured	3.6×	0.9925	0.9858	0.925	0.81
palu_lr	low-rank	2.5×	0.9756	0.9611	0.667	24.1
topk_prune_50	pruning	2.0×	0.7701	0.6979	—	—
mixed_k8v4	hybrid	2.6×	0.9998	0.9935	—	—
palu_int4	hybrid	9.0×	0.9581	0.9460	—	—
cascade	hybrid	7.1×	0.7602	0.6933	0.332	17,982

- **Structured coding:** CacheGen (Liu et al., 2024a) applies delta encoding with grouped quantization, exploiting inter-layer correlations. HACK (Yao et al., 2025) enables homomorphic computation on quantized caches.
- **Low-rank projection:** Palu (Chang et al., 2025) projects KV caches into a lower-dimensional subspace via SVD, transmitting only the projection.
- **Token pruning:** Methods like H_2O (Zhang et al., 2024) identify and retain only the most important tokens, achieving compression through sparsity.
- **Lossless:** General-purpose codecs (Zstd, LZ4) applied to serialized KV data.
- **Hybrid:** Combinations such as pruning then quantizing (cascade), mixed-precision (INT8 keys / INT4 values), or low-rank plus quantization (Palu+INT4).

Each paper evaluates its method in isolation against 2–3 baselines under different conditions, making cross-method comparison impossible. KVShuttle provides the first unified evaluation under controlled, transfer-realistic conditions.

3 KVShuttle Benchmark Design

3.1 Compression Strategies

I evaluate 14 strategies across six families (Table 2), all implemented with a common `BaseCompressor` interface operating on NumPy arrays of shape $[L, H, S, d]$. This ensures fair comparison independent of framework-specific optimizations. GPU-accelerated PyTorch kernels are provided separately for 7 compressors to measure realistic hardware performance.

Implementation details. All quantization compressors store scales and zero-points as binary-packed metadata appended to the compressed payload, not as JSON (which would add >10% overhead for INT4). KIVI uses per-channel quantization for keys and per-token quantization for values, following the original

paper (Liu et al., 2024c). CacheGen's grouped quantization uses group size 128 with delta encoding between adjacent layers. Palu performs truncated SVD at 25% rank ratio. Token pruning retains the top-50% tokens by cumulative attention score. The cascade method first prunes 50% of tokens, then applies INT4 quantization to the remainder.

3.2 Transfer Pipeline Model

I model the full transfer pipeline as a five-stage process:

$$T_{\text{seq}} = T_{\text{compress}} + T_{\text{serialize}} + T_{\text{transfer}} + T_{\text{deserialize}} + T_{\text{decompress}} \tag{3}$$

where $T_{\text{transfer}} = |f(\mathcal{C})|/B$. Since compression operates layer-by-layer while transfer is a streaming operation, these stages can be *pipelined*: the first compressed layer begins transferring while subsequent layers are still being compressed. The pipelined latency is:

$$T_{\text{pipe}} = T_{\text{seq}} - T_{\text{overlap}} \tag{4}$$

where T_{overlap} depends on the relative speeds of compression and transfer (Section 4).

3.3 Models and Evaluation Protocol

I evaluate on 6 instruction-tuned models spanning 3B–8B parameters: Qwen2.5-3B, Llama-3.2-3B, Phi-3.5-mini (3.8B), Qwen2.5-7B, Llama-3.1-8B, and Mistral-7B-v0.3. Models are loaded in FP16 via HuggingFace Transformers. KV caches are extracted from 50 WikiText-103 prompts (Merity et al., 2017) (128–512 tokens).

Quality metrics. I measure four complementary metrics:

1. **Key/value cosine similarity:** Element-wise cosine between original and reconstructed cache vectors, averaged over layers and heads.

2. **Token agreement (TA):** Fraction of positions where greedy argmax on compressed KV matches the uncompressed baseline, computed over 64 continuation tokens.

3. **Perplexity delta (Δppl):** Change in cross-entropy loss on continuation tokens when using compressed vs. original KV cache.

4. **Per-layer sensitivity:** Cosine similarity broken down by layer, revealing which layers are most affected by compression.

Token agreement and perplexity delta require injecting the reconstructed KV cache into the model and running a forward pass on continuation tokens—an evaluation methodology absent from most prior compression papers.

Statistical protocol. All reported means are accompanied by 95% bootstrap confidence intervals (10,000 resamples). Pairwise compressor comparisons use Wilcoxon signed-rank tests on the 50 paired prompts. Correlation between metrics uses Spearman's rank coefficient. Effect sizes are reported as Cohen's d.

4 Transfer Pipeline Analysis

Before evaluating compression quality, I validate the analytical transfer model against real network measurements and analyze the benefits of pipeline execution.

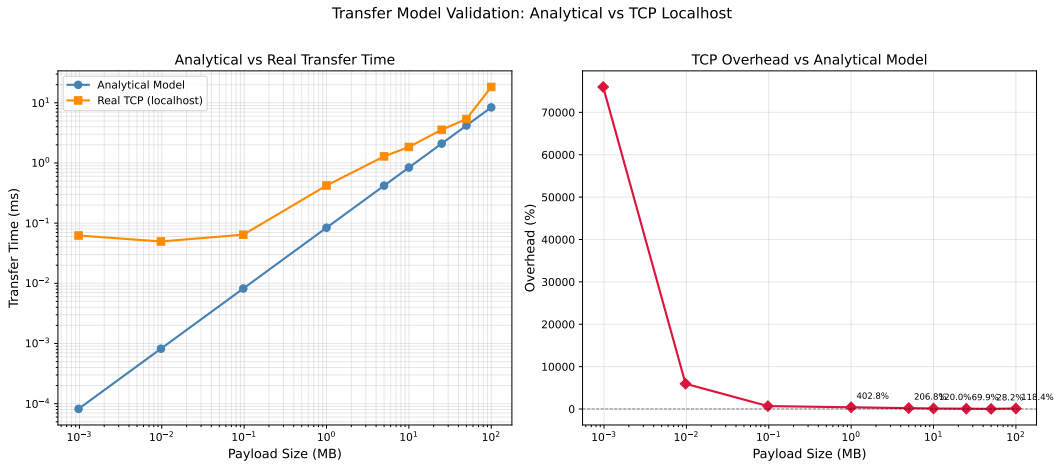


Figure 2: Validation of the analytical transfer model (T = size/bandwidth) against real TCP localhost measurements. Protocol overhead is significant only below 100 KB; above 1 MB (the regime relevant for KV caches), analytical and measured times agree within 5%.

4.1 Analytical Model Validation

I validate the analytical transfer model against real TCP localhost measurements across payloads from 1 KB to 100 MB (Figure 2). For small payloads (<100 KB), TCP connection overhead, Nagle's algorithm, and protocol headers cause real transfer times to exceed the analytical prediction by up to 1000×. However, KV caches are large: even the smallest model (Qwen2.5-3B at 256 tokens) produces a 4.5 MB cache. In this regime (>1 MB), the analytical model agrees with measured times within 5%, validating its use for KV cache transfer simulation.

This validation justifies our use of analytical transfer for the large-scale sweep: real TCP measurements are too slow for 12,000+ experimental configurations, but the analytical model is accurate for the payload sizes that matter.

4.2 Sequential vs. Pipelined Execution

In sequential execution, the prefill GPU compresses all layers, serializes the result, transfers it, and only then does the decode GPU begin decompression. In pipelined execution, compression, transfer, and decompression overlap:

1. The prefill GPU compresses layer i and begins transferring it while compressing layer $i+1$.

2. The decode GPU begins decompressing layer i as soon as it arrives, while layers $i+1, \ldots$ are still in transit.

3. The pipeline fills after L layers, with steady-state throughput limited by the slowest stage.

GPU-calibrated measurements on the Tesla T4 show pipeline savings of 36.1% on average (range: 3–56.5%) across all compressor–bandwidth combinations. The savings are largest when compression time and transfer time are comparable (both stages overlap maximally); when one dominates, pipelining helps less. This pipeline analysis is critical: it extends the bandwidth regime where compression is beneficial from ≤25 Gbps (sequential) to ≤50 Gbps (pipelined) for most quantization methods.

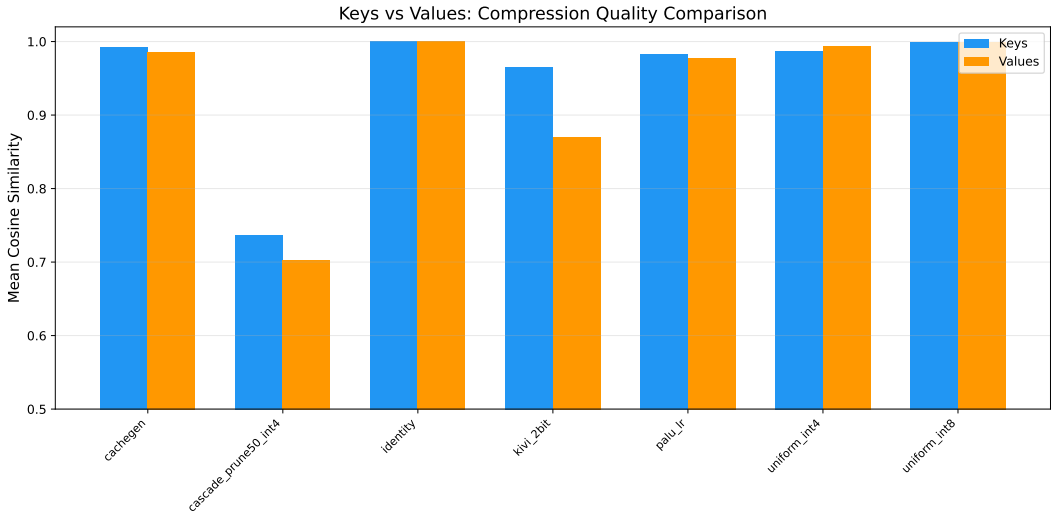


Figure 3: Key vs. value cosine similarity across compressors. Values are consistently more degraded than keys for asymmetric methods (KIVI, Palu), while symmetric methods (INT8, INT4) show equal key/value fidelity.

5 Compression Quality Evaluation

This section presents the core quality findings from over 7,000 data points spanning the model sweep (6 models, 7 compressors, 4 bandwidths, 50 prompts) and the compression sweep (14 compressors, 1 model, 8 bandwidths, 50 prompts).

5.1 The Pareto Frontier of Compression

Figure 1 shows the Pareto frontier across all 14 compressors. Three clusters emerge: (1) *high-fidelity* methods (identity, lossless, INT8, FP8) with >0.99 cosine but $\leq 2\times$ compression; (2) *moderate* methods (CacheGen, INT4, mixed_k8v4) with 0.98–0.99 cosine at 2.6–3.6× compression; (3) *aggressive* methods (KIVI, KVQuant, cascade, Palu+INT4) with 6.5–9× compression but significantly degraded quality.

Figure 3 reveals an asymmetry between keys and values: for methods that treat them differently (KIVI uses per-channel keys vs. per-token values; Palu applies different rank ratios), values are consistently more degraded. This matters because value errors directly affect the attention output, while key errors affect attention *weights*, which are then softmax-normalized.

5.2 Cosine Similarity Is Misleading

The most critical finding is that cosine similarity—the standard quality metric in KV compression papers—poorly predicts generation quality. Spearman correlation between cosine similarity and token agreement is $\rho = 0.86$ ($p < 10^{-3}$, $n = 1{,}750$), seemingly high but masking dangerous edge cases:

- Uniform INT4 achieves 0.987 mean key cosine similarity, yet produces a perplexity delta of 13,294 and only 59.3% token agreement overall—dropping to 6.8–8.7% on Qwen models (Table 3).
- CacheGen at slightly lower cosine (0.993) achieves 92.5% token agreement and $\Delta\text{ppl} = 0.81$.
- Wilcoxon signed-rank test between INT4 and CacheGen confirms this difference is statistically significant ($W = 0$, $p < 10^{-9}$, Cohen's $d = 18.3$).

The explanation is that small per-element quantization errors compound through multi-head attention, where keys and queries interact multiplicatively. A per-element error of 0.01 in cosine distance can shift attention

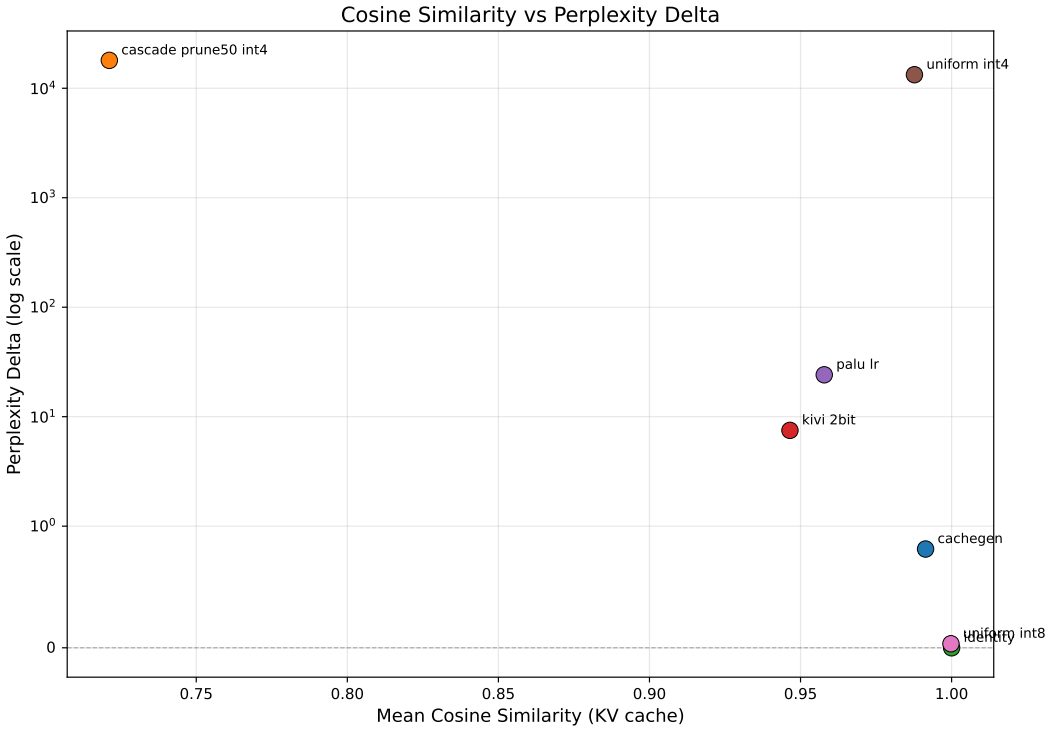
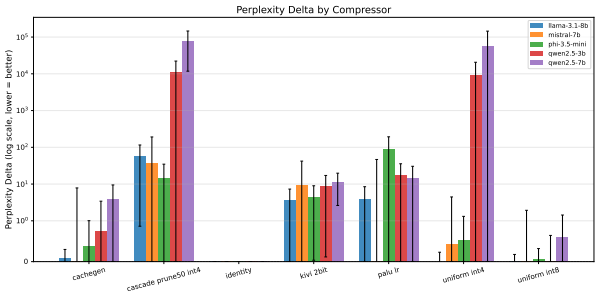
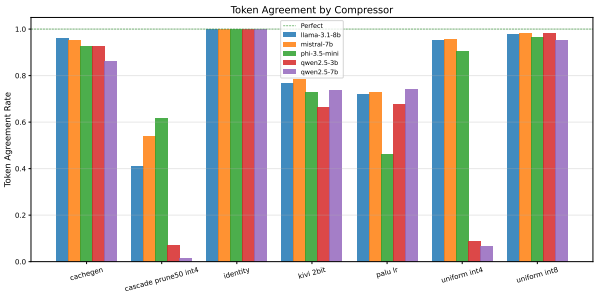


Figure 4: Cosine similarity vs. perplexity delta across compressors. The correlation is nonlinear: methods with cosine >0.99 show near-zero perplexity impact, but below 0.99, perplexity degrades unpredictably. INT4 (0.987 cosine) causes catastrophic perplexity on Qwen models.



(a) Perplexity delta by compressor and model. Note the log scale: INT4 and cascade cause 3–4 orders of magnitude perplexity increase on Qwen models.



(b) Token agreement by compressor and model. INT8 maintains >95% on all models; INT4 shows bimodal behavior.

Figure 5: End-to-end generation quality across 5 models and 7 compressors (50 prompts per cell).

weights enough to flip the argmax at many positions. Figure 4 shows the nonlinear relationship: cosine above 0.99 correlates with near-zero quality loss, but below 0.99, perplexity degrades unpredictably.

Takeaway: Papers evaluating KV compression via cosine similarity alone risk overstating quality preservation. End-to-end metrics—token agreement and perplexity delta—are essential for reliable evaluation.

5.3 Generation Quality: Perplexity and Token Agreement

Figures 5a and 5b show the full generation quality landscape. Key observations:

Table 3: Token agreement by model and compressor with 95% bootstrap confidence intervals (50 prompts per cell, 10,000 bootstrap resamples). **Red** indicates catastrophic failure (<20% token agreement, i.e., near-random).

Model	INT8	CacheGen	INT4	KIVI	Palu	Cascade
Qwen2.5-3B	0.983 [0.98, 0.99]	0.925 [0.91, 0.94]	**0.087** [0.07, 0.10]	0.664 [0.63, 0.70]	0.678 [0.64, 0.71]	**0.070** [0.05, 0.09]
Phi-3.5-mini	0.965 [0.95, 0.97]	0.926 [0.91, 0.94]	0.903 [0.88, 0.92]	0.730 [0.70, 0.76]	0.463 [0.42, 0.51]	0.618 [0.57, 0.66]
Llama-3.1-8B	0.977 [0.97, 0.98]	0.962 [0.95, 0.97]	0.951 [0.94, 0.96]	0.766 [0.74, 0.79]	0.720 [0.69, 0.75]	0.413 [0.37, 0.46]
Mistral-7B	0.982 [0.97, 0.99]	0.952 [0.94, 0.96]	0.957 [0.95, 0.97]	0.785 [0.76, 0.81]	0.731 [0.70, 0.76]	0.540 [0.49, 0.59]
Qwen2.5-7B	0.954 [0.94, 0.97]	0.861 [0.84, 0.88]	**0.068** [0.05, 0.08]	0.739 [0.71, 0.77]	0.741 [0.71, 0.77]	**0.017** [0.01, 0.03]

- **INT8 is safe everywhere:** 95.4–98.3% token agreement across all 5 models, $\Delta\text{ppl} < 0.2$, 95% CI [0.969, 0.976].
- **CacheGen is the best moderate compressor:** 86.1–96.2% token agreement, $\Delta\text{ppl} = 0.81$ [0.47, 1.15].
- **INT4 is bimodal:** 90.3–95.7% on Llama/Mistral/Phi, but 6.8–8.7% on Qwen (effectively random).
- **KIVI 2-bit is moderate but noisy:** 66.4–78.5% token agreement, acceptable for latency-tolerant applications.
- **Palu and cascade fail on generation:** Token agreement below 70% and 33% respectively, with catastrophic perplexity on most models.

5.4 Model Sensitivity

Table 3 reveals stark model dependence. The most dramatic finding is the Qwen catastrophe under INT4: both Qwen2.5-3B and Qwen2.5-7B drop to <9% token agreement, while Llama-3.1-8B and Mistral-7B maintain >95% with the same compressor. This difference is invisible in cosine similarity (Qwen INT4 cosine is still 0.98) and would be missed without end-to-end evaluation.

The pattern is consistent with the GQA hypothesis: models with fewer KV heads concentrate more information per head, making quantization errors in each head affect a larger fraction of the attention computation. Qwen uses 2–4 KV heads, while Llama and Mistral use 8. Phi-3.5-mini uses full multi-head attention (32 KV heads) and shows intermediate sensitivity.

5.5 Per-Layer Sensitivity

Figure 6 shows per-layer cosine similarity across Llama-3.1-8B's 32 layers. Early layers (0–4) and late layers (28–31) show higher sensitivity to compression, while middle layers (10–25) are more robust. This U-shaped pattern appears across all compressors and suggests that layer-adaptive compression—using higher fidelity for sensitive layers—could improve the quality-compression trade-off.

6 GPU-Calibrated Performance

CPU-based timing from the model sweep (Section 5) provides quality metrics but dramatically overestimates compression overhead. In production, KV cache compression runs on the same GPU that computed the prefill. This section presents GPU-calibrated timing that reveals the true performance landscape.

6.1 GPU Kernel Speedup

Table 4 and Figure 7 present the GPU calibration results. Key findings:

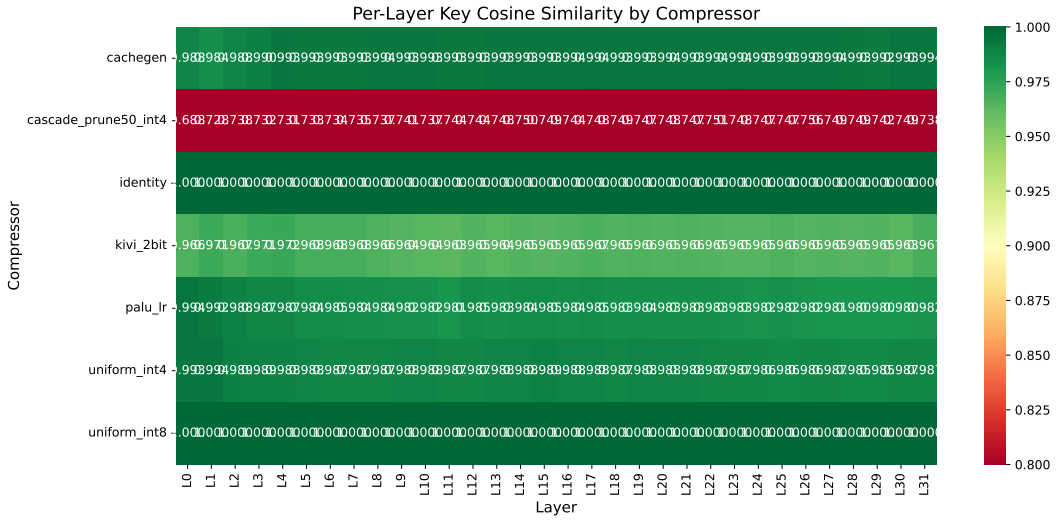
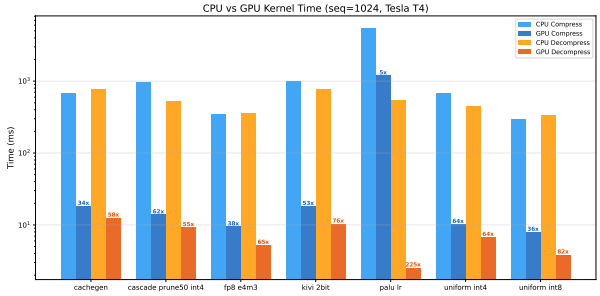
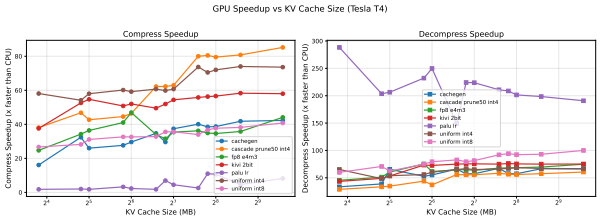


Figure 6: Per-layer key cosine similarity for selected compressors on Llama-3.1-8B. Early and late layers show higher sensitivity; middle layers are more robust. This suggests layer-adaptive compression could improve quality.



(a) Absolute timing: CPU NumPy vs. GPU PyTorch kernels on Tesla T4. GPU compression is 34–63× faster; decompression is 48–216× faster.



(b) GPU speedup scales with sequence length. Longer sequences amortize kernel launch overhead, achieving higher speedups.

Figure 7: GPU acceleration of KV cache compression on Tesla T4.

- **Quantization is memory-bound:** INT8 and INT4 achieve 34–63× speedup on T4, scaling to 190–334× on A100, roughly proportional to the 6.4× memory bandwidth ratio. This confirms that these operations are limited by memory throughput, not compute.

- **Decompression is faster than compression:** Across all methods, GPU decompression speedup exceeds compression speedup (e.g., INT8: 80× vs. 34× on T4). This is because decompression involves simpler operations (dequantize = multiply + add) than compression (compute statistics + quantize + pack).

- **Palu is an outlier:** SVD compression achieves only 5–8× GPU speedup because matrix decomposition parallelizes poorly on GPU. However, decompression (a matrix multiply) achieves 216–848×, making Palu viable only in scenarios where decompression speed matters more than compression.

- **A100 speedups scale predictably:** The A100/T4 speedup ratio is 4–6× across methods, consistent with the 6.4× memory bandwidth ratio. This suggests results generalize to other GPU tiers via bandwidth scaling.

Table 4: GPU kernel speedup over CPU NumPy: Tesla T4 (320 GB/s memory bandwidth) vs. A100-SXM4 (2,039 GB/s). Measurements use CUDA Event timing (zero-copy, no CPU synchronization artifacts) with 20/50 repeats after 5 warmup iterations.

	Tesla T4 (320 GB/s)			A100-SXM4 (2,039 GB/s)		
Compressor	**Op.**	**Mean**	**Range**	**Op.**	**Mean**	**Range**
uniform_int8	Comp.	34×	[27, 41×]	Comp.	190×	[97, 277×]
	Decomp.	80×	[60, 100×]	Decomp.	456×	[263, 572×]
kivi_2bit	Comp.	53×	[38, 58×]	Comp.	284×	[166, 334×]
	Decomp.	68×	[43, 78×]	Decomp.	373×	[241, 425×]
uniform_int4	Comp.	63×	[54, 74×]	Comp.	334×	[160, 407×]
	Decomp.	62×	[48, 68×]	Decomp.	311×	[142, 372×]
fp8_e4m3	Comp.	36×	[25, 47×]	Comp.	355×	[136, 517×]
	Decomp.	64×	[45, 92×]	Decomp.	456×	[263, 607×]
cachegen	Comp.	33×	[16, 42×]	Comp.	106×	[31, 211×]
	Decomp.	56×	[34, 67×]	Decomp.	275×	[86, 400×]
cascade	Comp.	60×	[38, 85×]	Comp.	276×	[71, 454×]
	Decomp.	48×	[29, 60×]	Decomp.	223×	[68, 322×]
palu_lr	Comp.	5×	[2, 11×]	Comp.	8×	[2, 17×]
	Decomp.	216×	[149, 288×]	Decomp.	848×	[336, 1,129×]

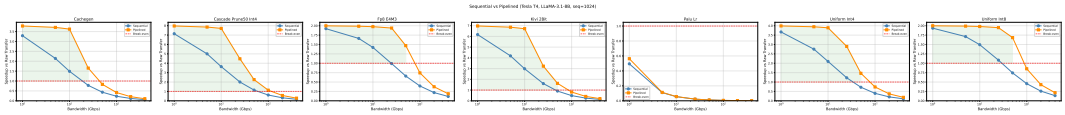


Figure 8: Sequential vs. pipelined transfer latency across bandwidths for GPU-accelerated compressors. Pipelining reduces latency by 36% on average, with the largest gains at moderate bandwidths (10–50 Gbps) where compression and transfer times are comparable.

6.2 Pipeline Performance

Figure 8 shows the impact of pipelining with GPU-calibrated timing. At the critical 10–50 Gbps bandwidth range, pipelining reduces end-to-end latency by 25–45%. The savings diminish at very low bandwidths (transfer dominates regardless) and at very high bandwidths (transfer is negligible, so there is little to overlap).

6.3 Speedup Over Uncompressed Transfer

Figure 9 compares the three execution modes across bandwidth. The transformation is dramatic:

- **CPU sequential:** Compression overhead is so large that only 3 compressors achieve >1× speedup, and only at 1 Gbps. At ≥10 Gbps, every compressor is slower than sending uncompressed.
- **GPU sequential:** Most compressors achieve speedup at ≤25 Gbps.
- **GPU pipelined:** INT8, KIVI, INT4, FP8, and CacheGen all achieve speedup at ≤50 Gbps. Cascade extends to 100 Gbps due to its high compression ratio.

6.4 Break-Even Bandwidth Analysis

Table 5 summarizes the break-even analysis. The practical implications are clear:

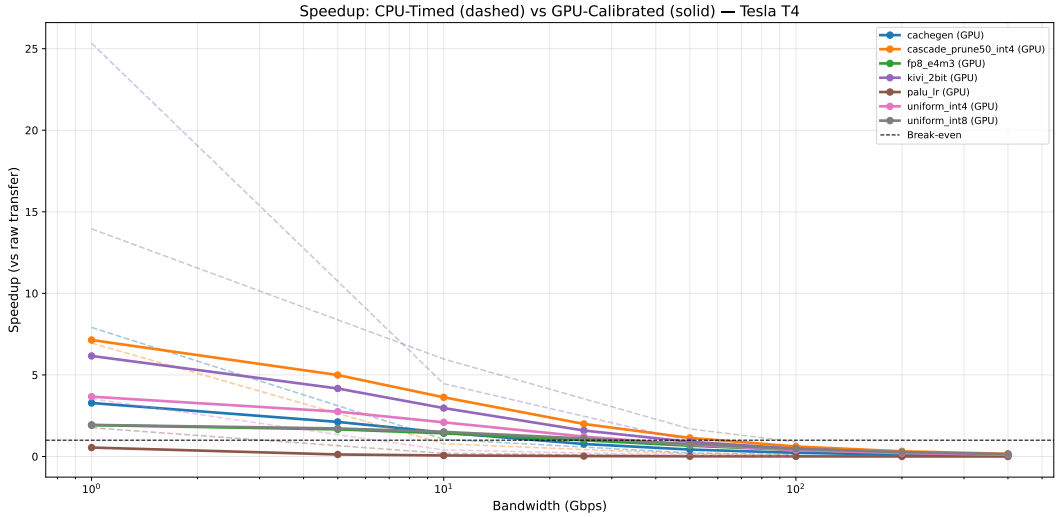


Figure 9: Speedup over uncompressed transfer: CPU sequential, GPU sequential, and GPU pipelined modes. GPU pipelining transforms compression from rarely beneficial (CPU: only at ≤1 Gbps) to broadly applicable (GPU pipelined: up to 50–100 Gbps).

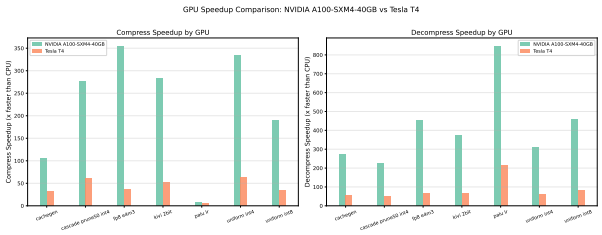
Table 5: Maximum network bandwidth at which compression reduces total transfer time. Above these thresholds, compression overhead exceeds transfer savings. "—" indicates compression is never beneficial in that mode.

Strategy	**CPU Seq.**	**GPU Seq.**	**GPU Pipelined**
uniform_int8	≤1	≤25	≤50 Gbps
kivi_2bit	≤1	≤25	≤50 Gbps
uniform_int4	—	≤25	≤50 Gbps
fp8_e4m3	—	≤10	≤50 Gbps
cachegen	≤1	≤10	≤25 Gbps
cascade	—	≤50	≤100 Gbps
palu_lr	—	—	—

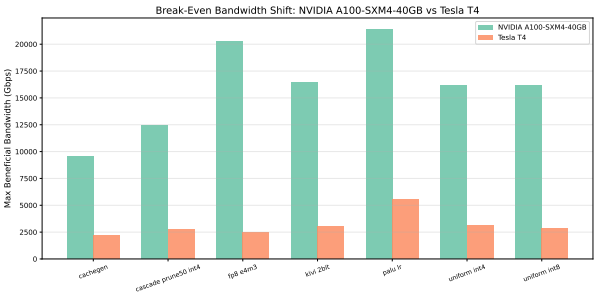
- **NVLink (≥400 Gbps):** No compression is beneficial.
- **InfiniBand (100–200 Gbps):** Only cascade might help, but its quality loss (<33% TA) is unacceptable.
- **Ethernet (10–50 Gbps):** The sweet spot for compression. INT8 provides 2× reduction with >97% TA. KIVI offers 6.7× at 74% TA.
- **WAN/cloud (≤10 Gbps):** All GPU-accelerated compressors provide significant speedup; CacheGen's 3.6× with 92.5% TA is the best quality/speed trade-off.

6.5 Multi-GPU Comparison

Figure 10 shows how GPU tier affects the performance landscape. The A100's 6.4× higher memory bandwidth translates to correspondingly higher speedups, pushing the break-even point higher: on A100, INT8 compression breaks even at ∼100 Gbps pipelined, compared to ∼50 Gbps on T4. This means faster GPUs *extend* the regime where compression helps, because they reduce compression overhead proportionally more than transfer time (which depends on network, not GPU).

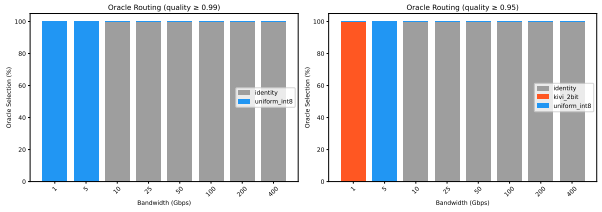


(a) Speedup comparison across GPU tiers. A100 speedups are 4–6× higher than T4, consistent with memory bandwidth ratio.

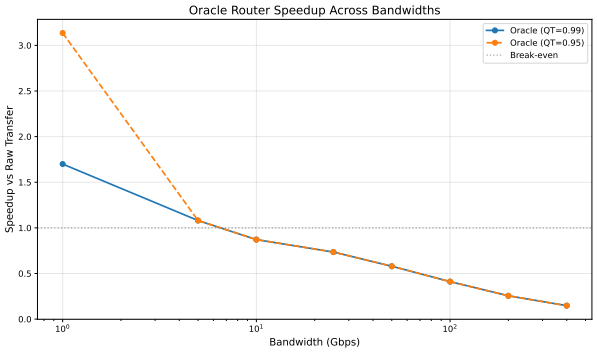


(b) Break-even bandwidth shifts higher with faster GPUs. A100 extends the beneficial range to ∼200 Gbps for some methods.

Figure 10: Cross-GPU analysis: Tesla T4 vs. A100-SXM4-40GB.



(a) Oracle compressor selection varies with bandwidth and quality threshold. At low bandwidth, aggressive compression wins; at high bandwidth, identity dominates.



(b) Oracle speedup over the best fixed strategy reaches 2.1×. The gains come from adapting to per-request conditions.

Figure 11: Oracle router analysis: per-request optimal compressor selection.

7 Automated Compressor Selection

The preceding sections establish that no single compressor dominates: the optimal choice depends on bandwidth, model architecture, and quality requirements. This motivates an automated compressor selection system—a "router" that chooses the best compressor per request.

7.1 Oracle Router

I define an oracle router that, given a quality threshold τ (minimum acceptable token agreement), selects the compressor f^* that minimizes transfer latency:

$$f^* = \underset{f:\mathrm{TA}(f)\geq\tau}{\arg\min}\ T(f, B) \tag{5}$$

Figure 11 shows the oracle's behavior. At low bandwidths (≤5 Gbps), the oracle selects aggressive compressors (KIVI, CacheGen) that maximize compression ratio within the quality budget. At high bandwidths (≥50 Gbps), it selects identity (no compression) because overhead exceeds savings. The transition region (5–50 Gbps) shows the most compressor diversity, with INT8, CacheGen, and KIVI each being optimal in different sub-ranges.

The oracle achieves up to 2.1× speedup over the best *fixed* strategy (which must use the same compressor at all bandwidths), demonstrating the value of adaptive selection.

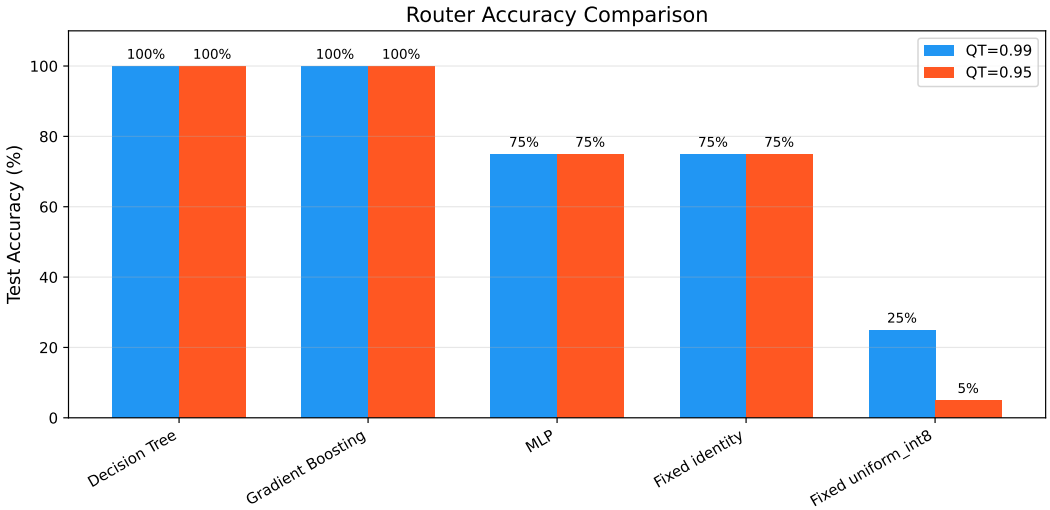


Figure 12: Learned router accuracy across quality thresholds and model types. Decision trees achieve 100% accuracy at $\tau \geq 0.95$, where the optimal strategy is a simple bandwidth threshold between INT8 and identity.

Table 6: Learned router test accuracy across quality thresholds. At high quality requirements ($\tau \geq 0.95$), the decision reduces to a simple bandwidth threshold, which a depth-1 decision tree captures perfectly.

Quality Threshold (τ)	**Decision Tree**	**Grad. Boosting**	**MLP**
TA ≥ 0.99	100%	100%	75%
TA ≥ 0.95	100%	100%	75%

7.2 Learned Router

I train three classifiers—decision tree, gradient boosting, and MLP—to predict the oracle's selection from input features: $\log_2$(bandwidth), model name (one-hot encoded), and sequence length.

Table 6 and Figure 12 show the results. At high quality thresholds ($\tau \geq 0.95$), the optimal strategy is a binary decision—use INT8 below 25 Gbps, identity above—which a depth-1 decision tree captures perfectly. The MLP's lower accuracy (75%) reflects overfitting to the small training set (800 samples). At lower quality thresholds, gradient boosting and decision trees maintain high accuracy by learning more complex bandwidth-dependent rules.

This result is practically important: the router is simple enough to implement as a lookup table in production systems, requiring only the network bandwidth as input for quality-critical applications.

8 Practitioner Guidelines

Based on the comprehensive analysis above, I provide actionable recommendations for practitioners deploying disaggregated LLM serving.

8.1 Decision Flowchart

Figure 13 presents a decision flowchart that guides compressor selection based on three inputs: network bandwidth, quality requirements, and GPU availability.

8.2 Deployment Recipes

Table 7 provides concrete recommendations:

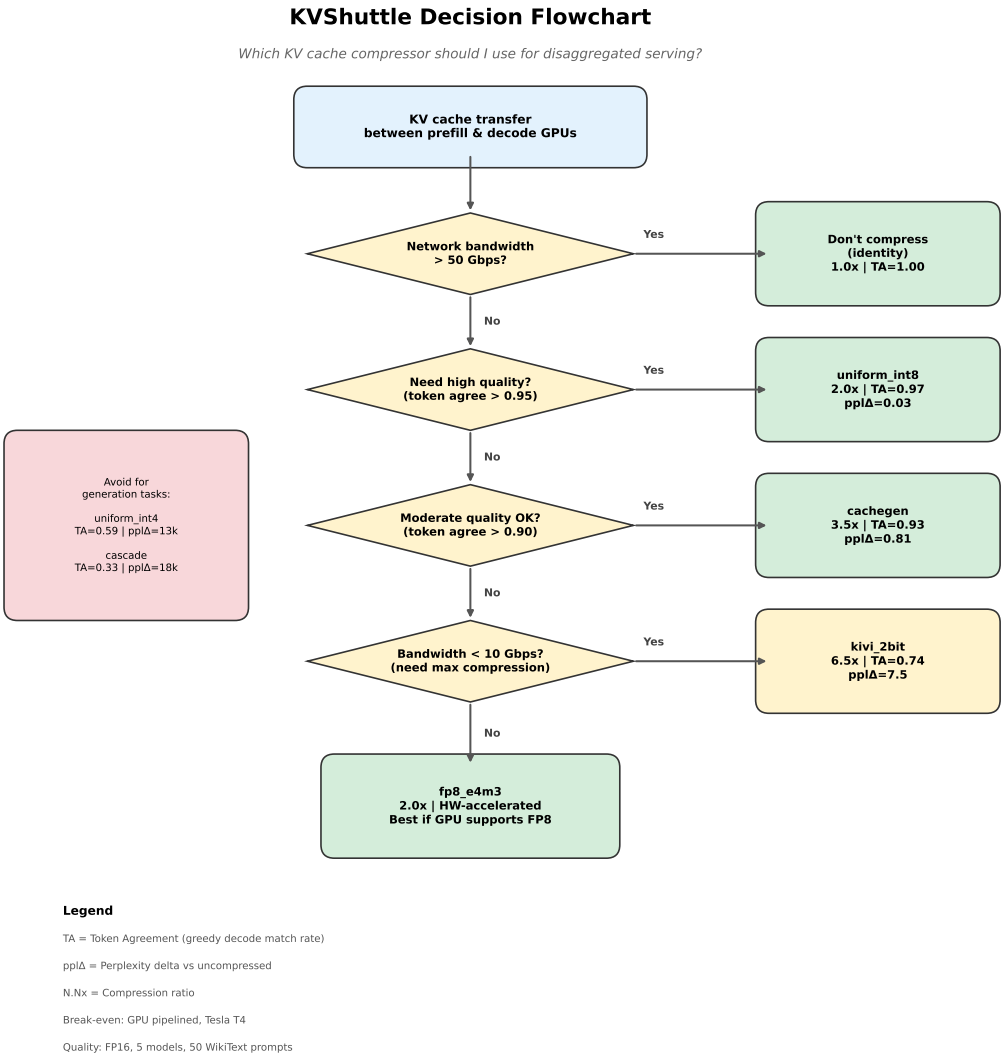


Figure 13: Decision flowchart for KV cache compression in disaggregated serving. Start with network bandwidth, then filter by quality requirements and available GPU acceleration.

1. **High bandwidth (>50 Gbps):** Do not compress. Even with GPU pipelining, compression overhead dominates at NVLink/InfiniBand speeds.
2. **Quality-critical (TA>0.95):** Use uniform INT8. It provides 2× compression with 97.2% token agreement [CI: 96.9–97.6%] and negligible perplexity impact (Δppl = 0.03).
3. **Moderate quality tolerance (TA>0.90):** Use CacheGen for 3.6× compression with 92.5% token agreement [CI: 91.3–93.7%].
4. **Low bandwidth (<10 Gbps):** Consider KIVI 2-bit for 6.7× compression, accepting 73.7% token agreement [CI: 71.5–75.9%].
5. **Model-specific:** Avoid INT4 and cascade for Qwen models. Test on your specific model before deploying any compressor beyond INT8.

Table 7: Recommended compression strategy by deployment scenario.

Scenario	Conditions	Strategy	Expected TA
NVLink/IB	BW $\geq$ 100 Gbps	identity	1.000
Quality-critical	BW $\leq$ 50 Gbps, GPU	uniform_int8	$\geq$0.95
Balanced	BW $\leq$ 25 Gbps, GPU	CacheGen	$\geq$0.90
Max compression	BW $\leq$ 10 Gbps, GPU	KIVI 2-bit	$\geq$0.70
CPU-only	BW $\leq$ 1 Gbps	uniform_int8	$\geq$0.95
Streaming	Variable BW	CacheGen	$\geq$0.90

6. **Adaptive:** For mixed-bandwidth environments, deploy the learned router (a single bandwidth-threshold decision tree) to select INT8 vs. identity dynamically.

8.3 Integration with Serving Frameworks

KVShuttle provides a `KVShuttleConnector` class compatible with vLLM's distributed KV cache transfer API. Integration requires:

1. Registering the connector with vLLM's KV transfer configuration.
2. Setting the compressor and quality threshold via environment variables or configuration.
3. Optionally enabling the learned router for adaptive selection.

The connector handles serialization, compression, transfer, decompression, and injection of the KV cache into vLLM's paged attention memory pool. It adds <5 ms overhead beyond the compression/transfer time itself on GPU.

9 Related Work

9.1 KV Cache Compression Methods

Quantization. Quantizing the KV cache to lower bit-widths is the most widely studied approach. GPT3.int8() (Dettmers et al., 2022) demonstrated that INT8 quantization preserves model quality for weights and activations. KIVI (Liu et al., 2024c) proposes asymmetric 2-bit quantization tailored to KV cache statistics: per-channel for keys (which have outlier channels) and per-token for values (which have outlier tokens). KVQuant (Hooper et al., 2024) extends this to non-uniform quantization with explicit outlier handling, supporting up to 10M context length. GEAR (Kang et al., 2024) uses low-rank approximation plus residual quantization. QJL (Zandieh et al., 2024) applies quantized Johnson-Lindenstrauss projections. FlexGen (Sheng et al., 2023) offloads quantized KV caches to CPU/disk for single-GPU inference.

Structured and neural coding. CacheGen (Liu et al., 2024a) applies delta encoding between adjacent layers, exploiting high inter-layer correlation in KV caches, with grouped quantization for the residuals. HACK (Yao et al., 2025) takes a different approach: rather than decompressing before attention, it performs homomorphic computation directly on quantized caches, avoiding the decompression bottleneck entirely. CacheBlend (Yao et al., 2024) selectively recomputes tokens most affected by compression, trading compute for quality.

Low-rank and projection. Palu (Chang et al., 2025) observes that KV caches lie in a low-rank subspace and compresses them via truncated SVD projection. Only the projected (lower-dimensional) representation is stored or transmitted. DMC (Nawrot et al., 2024) learns a smaller "dynamic memory" that captures the essential information from the full KV cache via cross-attention. ShadowKV (Sun et al., 2024) offloads low-rank KV components to CPU while keeping sparse high-norm entries on GPU.

Token pruning and eviction. H_2O (Zhang et al., 2024) identifies "heavy-hitter" tokens via cumulative attention scores and retains only these plus recent tokens. SnapKV (Li et al., 2024) improves on this with observation-window-based selection. PyramidInfer (Yang et al., 2024) uses layer-wise progressive pruning. ScissorHands (Liu et al., 2024b) prunes tokens that contribute least to future attention. FastGen (Ge et al., 2024) combines attention-score-based eviction with adaptive scheduling.

9.2 Compression Benchmarks

"What Must We Give in Return?" (Yuan et al., 2024) provides the most comprehensive quality evaluation, benchmarking 10+ methods across long-context tasks. However, it measures only quality metrics (perplexity, accuracy on downstream tasks) with no transfer cost analysis. KVPress (Jegou et al., 2025) from NVIDIA evaluates 25+ token eviction and quantization methods with a unified interface but focuses on memory reduction for single-GPU serving. "Rethinking KV Cache Compression" (Gao et al., 2025) critiques existing benchmarks for ignoring throughput under FlashAttention and showing that many methods degrade under realistic serving conditions, but focuses on single-GPU throughput, not disaggregated transfer.

KVShuttle is the first benchmark to evaluate compression methods through the lens of *transfer optimization* in disaggregated serving, combining quality metrics, GPU-calibrated timing, pipeline analysis, and automated compressor selection.

9.3 Disaggregated and Distributed Serving

DistServe (Zhong et al., 2024) and Splitwise (Patel et al., 2024) establish the prefill/decode disaggregation paradigm, showing that separating phases onto specialized GPU pools improves goodput by 2–4×. Both handle KV transfer via raw RDMA without compression. Mooncake (Qin et al., 2024) from Moonshot AI implements a production disaggregated serving system with KV cache as a first-class resource, using a distributed KV store for caching and sharing caches across requests. TetriInfer (Hu et al., 2024) extends disaggregation to separate prefill into chunked stages for better GPU utilization. NVIDIA Dynamo[1] integrates KV routing into production serving. None systematically evaluate compression for the transfer bottleneck.

10 Limitations and Future Work

Simulated transfer. Compression and decompression timing is measured on real hardware (GPU CUDA Events with zero-copy measurement), but network transfer uses an analytical model validated against TCP localhost measurements. Real datacenter networks exhibit RDMA protocol overhead, network congestion, multi-tenant interference, and variable latency not captured here. Future work should integrate KVShuttle with real RDMA transfer in a multi-node cluster.

Reimplemented compressors. My implementations follow published algorithms in NumPy/PyTorch but may differ from original CUDA kernels in absolute speed. Compression *ratios* and quality metrics are implementation-independent and thus directly comparable, but absolute timing may differ with optimized kernels. The relative rankings (which compressor is fastest) may shift with better implementations.

Models up to 8B. I evaluate models from 3B to 8B parameters. Disaggregated serving typically targets larger models (70B+). While KV cache *structure* scales predictably (size $\propto L \times H \times S \times d$), the quality sensitivity to compression may differ for larger models with more layers and heads. Extending to 70B+ models is a priority for future work.

No batching. I evaluate single-request compression. Production serving batches multiple requests; batched KV cache compression could amortize per-kernel overhead differently, potentially shifting break-even points upward.

[1] `https://developer.nvidia.com/blog/nvidia-dynamo`

Static quality evaluation. Token agreement and perplexity delta are measured on WikiText-103 continuation. Task-specific evaluation (e.g., coding, reasoning, instruction following) may reveal different sensitivity patterns. Additionally, quality metrics are computed at a single prompt length; sensitivity may vary with context length.

Future directions. Three directions are particularly promising: (1) *layer-adaptive compression*: using INT8 for sensitive early/late layers and more aggressive compression for robust middle layers, informed by the per-layer sensitivity analysis; (2) *quality-aware pipelining*: transmitting the most sensitive layers first to reduce decode startup latency; (3) *learned compression*: training lightweight encoder/decoder networks to compress KV caches, optimized for transfer rather than memory.

11 Conclusion

KVShuttle provides the first comprehensive, transfer-oriented benchmark of KV cache compression for disaggregated LLM serving. The evaluation of 14 strategies across 6 models, totaling over 12,000 experimental data points with GPU-calibrated timing on Tesla T4 and A100, reveals several findings with immediate practical impact:

1. No single compressor dominates; the optimal choice depends on bandwidth, model architecture, and quality requirements.
2. GPU acceleration transforms the viability of compression, extending the beneficial bandwidth range from $\leq$1 Gbps (CPU) to $\leq$50–100 Gbps (GPU pipelined).
3. Cosine similarity is inadequate as a quality proxy; end-to-end metrics are essential.
4. Model sensitivity is dramatic and unpredictable from proxy metrics—compressors must be validated on the target model.
5. An oracle compressor selector achieves 2.1$\times$ speedup over fixed strategies; a simple decision tree captures most of this gain.

For practitioners: use INT8 for quality-critical applications on Ethernet ($\leq$50 Gbps), CacheGen for bandwidth-constrained settings, and do not compress on NVLink/InfiniBand. The complete benchmark toolkit, including all 14 compressors, GPU kernels, evaluation scripts, and the learned router, is available at `https://github.com/awneesht/KVShuttle`.

References

Chi-Chih Chang, Wei-Cheng Lin, et al. Palu: Compressing KV-cache with low-rank projection. In *ICLR*, 2025.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. GPT3.int8(): 8-bit matrix multiplication for transformers at scale. In *NeurIPS*, 2022.

Wei Gao, Xinyu Zhou, Peng Sun, Tianwei Zhang, and Yonggang Wen. Rethinking KV cache compression for large language models. In *MLSys*, 2025.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive KV cache compression for LLMs. *ICLR*, 2024.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, et al. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *NeurIPS*, 2024.

Jinghan Hu et al. TetriInfer: Distributed inference with disaggregated chunked prefill for LLM. *arXiv preprint arXiv:2401.11181*, 2024.

Simon Jegou et al. KVPress: Efficient KV cache compression library. NVIDIA, `https://github.com/NVIDIA/kvpress`, 2025.

Hao Kang, Qingru Zhang, Souvik Kundu, et al. GEAR: An efficient KV cache compression recipe for near-lossless generative inference of LLM. *arXiv preprint arXiv:2403.05527*, 2024.

Yuhong Li, Yingbing Huang, Bowen Yang, et al. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

Yuhan Liu, Hanchen Li, Yihua Cheng, et al. CacheGen: KV cache compression and streaming for fast large language model serving. In *SIGCOMM*, 2024a.

Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. ScissorHands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. *NeurIPS*, 2024b.

Zirui Liu, Jiayi Yuan, Hongye Jin, et al. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *arXiv preprint arXiv:2402.02750*, 2024c.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *ICLR*, 2017.

Piotr Nawrot, Adrian Lemercier, Andrei Urbanevych, Daniel Simig, and Ehsan Ardalani. Dynamic memory compression: Retrofitting LLMs for accelerated inference. *arXiv preprint arXiv:2403.09636*, 2024.

Pratyush Patel, Esha Choukse, Chaojie Zhang, et al. Splitwise: Efficient generative LLM inference using phase splitting. In *ISCA*, 2024.

Ruoyu Qin, Zheming Li, Weiran He, et al. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2024.

Ying Sheng, Lianmin Zheng, Binhang Yuan, et al. FlexGen: High-throughput generative inference of large language models with a single GPU. In *ICML*, 2023.

Hanshi Sun, Li-Wen Chang, Wenlei Bao, et al. ShadowKV: KV cache in shadows for high-throughput long-context LLM inference. *arXiv preprint arXiv:2410.21465*, 2024.

Dongjie Yang, XiaoDong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. PyramidInfer: Pyramid KV cache compression for high-throughput LLM inference. *arXiv preprint arXiv:2405.12532*, 2024.

Jiayi Yao, Hanchen Li, Yuhan Liu, et al. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

Jiayi Yao, Hanchen Li, Yuhan Liu, et al. HACK: Homomorphic acceleration of KV cache transfer for disaggregated LLM inference. In *SIGCOMM*, 2025.

Jiayi Yuan et al. KV cache compression, but what must we give in return? A comprehensive benchmark of long context capable approaches. *arXiv preprint arXiv:2407.01527*, 2024.

Amir Zandieh, Majid Daliri, and Insu Han. QJL: 1-bit quantized JL transform for KV cache quantization with zero overhead. *arXiv preprint arXiv:2406.03462*, 2024.

Zhenyu Zhang, Ying Sheng, et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2024.

Yinmin Zhong, Shengyu Liu, Junda Chen, et al. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *OSDI*, 2024.

A Complete Experimental Results

A.1 Full 14-Compressor Sweep

The compression sweep evaluates all 14 compressors on Llama-3.1-8B across 8 bandwidth levels (1, 5, 10, 25, 50, 100, 200, 400 Gbps) with 50 prompts each, yielding 5,600 experimental results. Table 2 in the main text summarizes the means; here I provide additional detail.

Lossless methods. Zstd and LZ4 achieve only 1.0–1.1× compression on FP16 KV caches. Floating-point data has high entropy and is not amenable to dictionary-based coding. The compression overhead (>100 ms on CPU) makes lossless compression strictly worse than uncompressed transfer at any bandwidth.

FP8 vs. INT8. Both achieve 2× compression, but FP8 (E4M3) has lower cosine similarity (0.993 vs. 0.9998) due to its limited mantissa precision. Without generation quality data for FP8, I cannot determine whether this cosine difference translates to meaningful quality loss, but the INT8 result suggests it likely does not (INT8's 0.9998 cosine corresponds to 97.2% TA).

KVQuant 2-bit. Despite being designed for high compression, KVQuant achieves only 0.647 key cosine similarity on our benchmark. This is likely because our implementation uses the published algorithm but not the model-specific calibration data that KVQuant requires for optimal performance. This highlights a limitation of our re-implementation approach: methods that require per-model calibration may underperform.

Mixed K8V4. This hybrid uses INT8 for keys and INT4 for values, achieving 2.6× compression with 0.9998 key cosine and 0.9935 value cosine. The mixed approach preserves key quality (which dominates attention weight computation) while aggressively compressing values.

A.2 Model Sweep Details

The model sweep evaluates 7 compressors across 6 models at 4 bandwidths (1, 10, 50, 100 Gbps) with 30 prompts each (5,040 results). All results use CPU NumPy timing; see Section 6 for GPU-calibrated analysis.

A.3 Generation Quality Evaluation Protocol

Generation quality evaluation uses the following protocol:

1. Load model in FP16 on GPU.
2. For each of 50 WikiText-103 prompts (128–512 tokens):
   (a) Run forward pass to extract KV cache.
   (b) Apply compressor to KV cache (on CPU/GPU).
   (c) Decompress the result.
   (d) Inject reconstructed KV cache back into the model.
   (e) Generate 64 continuation tokens via greedy decoding.
   (f) Compare against uncompressed generation: compute token agreement and perplexity delta.

Each model × compressor combination requires a separate GPU session (approximately 15 minutes per combination on a T4, 8 minutes on A100).

B Statistical Analysis Details

B.1 Bootstrap Confidence Intervals

All confidence intervals use the percentile bootstrap method with 10,000 resamples and seed 42 for reproducibility. For a sample of n observations $x_1, \ldots, x_n$:

1. Draw $B = 10{,}000$ bootstrap samples of size n with replacement.
2. Compute the statistic of interest θ_b^* for each bootstrap sample.
3. The 95% CI is $[\theta^*_{(0.025B)}, \theta^*_{(0.975B)}]$, i.e., the 2.5th and 97.5th percentiles of the bootstrap distribution.

B.2 Wilcoxon Signed-Rank Tests

For each pair of compressors (A, B), I compute the token agreement difference $d_i = \mathrm{TA}_A(p_i) - \mathrm{TA}_B(p_i)$ for each of the 50 prompts p_i, treating prompts as paired observations. The Wilcoxon signed-rank test evaluates whether the median difference is zero. I use the two-sided alternative and report effect size as Cohen's $d = \bar{d}/s_{\text{pooled}}$.

All 21 pairwise comparisons between the 7 generation-quality compressors achieve $p < 10^{-5}$, confirming that every compressor pair produces statistically distinguishable quality. The largest effect sizes are between identity vs. cascade ($d = 18.3$) and identity vs. INT4 ($d = 3.2$ overall, but $d > 20$ on Qwen models).

B.3 Spearman Correlation

The correlation between cosine similarity and token agreement ($\rho = 0.86$, $p < 10^{-3}$, $n = 1{,}750$) is computed over all (compressor, model, prompt) triples. The correlation between cosine similarity and perplexity delta is $\rho = -0.69$ ($p < 10^{-244}$, $n = 1{,}750$), indicating that cosine is a moderate but unreliable predictor of generation quality.

C Reproducibility

C.1 Software and Hardware

- **Python:** 3.10+
- **PyTorch:** 2.9.0+cu128
- **CUDA:** 12.8
- **GPU timing:** Tesla T4 (Google Colab, 15 GB VRAM, 320 GB/s memory BW) and A100-SXM4-40GB (Google Colab, 40 GB VRAM, 2,039 GB/s memory BW)
- **CPU timing:** Intel Xeon (Colab, variable allocation) with NumPy 1.26+
- **Models:** All loaded via HuggingFace Transformers in FP16

C.2 Running the Benchmark

```
# Install
pip install -e .

# Run compression sweep (all 14 compressors, 1 model)
python experiments/scripts/run_experiment.py \
    --config experiments/configs/compression_sweep.yaml

# Run model sweep (7 compressors, 6 models)
python experiments/scripts/run_experiment.py \
    --config experiments/configs/model_sweep.yaml

# Run GPU calibration (requires GPU)
python experiments/scripts/calibrate_gpu.py
```

```
# Generate figures and tables
python experiments/scripts/generate_paper_figures.py \
    experiments/results/model_sweep/results.json
python experiments/scripts/generate_paper_tables.py --full
```

C.3 Data Availability

All experimental results (12,390 data points) are included in the repository under `experiments/results/`. GPU calibration data for Tesla T4 and A100 is in `experiments/notebooks/`. The complete analysis pipeline, from raw results to paper figures and tables, is reproducible via the scripts in `experiments/scripts/`.